LINUX COMMAND LINE · PRACTICE SET

Linux CLI Exercises

Ten exercises and a mini challenge, each laid out as **problem → command → output**.
Every output on these pages was produced by actually running the command.

11	45	bash
EXERCISES	TASKS SOLVED	GNU COREUTILS

```
$ grep -c "ERROR" app.log
3
```

How to read each task

The dark line is the command to type after the `$` prompt. The light box underneath is exactly what it printed. Short notes explain options that are easy to forget, and each exercise ends with the mistakes made on the first attempt and how they were fixed.

#	EXERCISE	COMMANDS	WHERE IT SHOWS UP IN DV WORK
01	**grep + wc**	`grep` `wc`	Counting failures in a simulation log: `grep -c "UVM_ERROR" sim.log`
02	**sort + uniq**	`sort` `uniq` `awk`	Ranking which error IDs or failing tests appear most often across a regression.
03	**find**	`find` `wc` `sort`	Locating every `sim.log`, coverage database or waveform dump under a regression run area.
04	**cut**	`cut`	Pulling one column (test name, seed, status) out of a delimited regression report.
05	**sed**	`sed` `cat`	Patching a define, parameter or path across many testbench and config files at once with `sed -i`.
06	**awk**	`awk`	Summing or averaging numbers from logs: simulation time, coverage %, errors per test.
07	**Process management**	`pgrep` `kill` `killall`	Finding and killing a hung simulation or a stuck regression job.
08	**Permissions**	`chmod` `stat`	Making run scripts executable (`chmod u+x run_sim.sh`) and keeping golden reference files read-only.
09	**Pipelines**	`sort` `uniq` `awk`	One-line triage: which test or seed fails most often in a regression.
10	**Log analysis**	`grep` `tail`	This is everyday simulation-log triage: swap `ERROR` for `UVM_ERROR` and `app.log` for `sim.log`.
11	**Mini challenge: sales report**	`sort` `awk` `head`	Turning a regression summary into numbers: pass rate, slowest tests, average runtime.

EXERCISE 01

grep + wc

Search and count lines

INPUT FILE

```
cat > students.txt << EOF
John Passed
Mary Failed
Alex Passed
Sarah Passed
David Failed
EOF
```

TASKS

1 Show only students who passed.

```
$ grep -i "passed" students.txt
```

```
John Passed
Alex Passed
Sarah Passed
```

2 Show only students who failed.

```
$ grep -i "failed" students.txt
```

```
Mary Failed
David Failed
```

3 Count how many students passed.

```
$ grep -ic "passed" students.txt
```

```
3
```

> **Note** Same result as `grep -i passed students.txt | wc -l`; `-c` lets grep do the counting itself.

4 Count the total lines in the file.

```
$ wc -l < students.txt
```

```
5
```

> **Note** Redirecting with `<` prints only the number. `wc -l students.txt` prints `5 students.txt`.

DEBUG NOTES · WHAT WENT WRONG FIRST

- The first heredoc was typed into `students.tx` (typo) and closed with lowercase `eof`, so the terminal kept waiting for input. The closing word must match exactly: `EOF`.

EXERCISE 02

sort + uniq

Order, de-duplicate and count

INPUT FILE

```
cat > fruits.txt << EOF
apple
banana
apple
orange
banana
apple
grape
EOF
```

TASKS

1 Sort the file.

```
$ sort fruits.txt
```

```
apple
apple
apple
banana
banana
grape
orange
```

2 Show unique fruit names.

```
$ sort -u fruits.txt
```

```
apple
banana
grape
orange
```

> **Note** Equivalent to `sort fruits.txt | uniq`.

3 Count occurrences of each fruit.

```
$ sort fruits.txt | uniq -c
```

```
      3 apple
      2 banana
      1 grape
      1 orange
```

4 Show the most repeated fruit.

```
$ sort fruits.txt | uniq -c | sort -nr | awk 'NR==1 {max=$1} $1==max'
```

```
      3 apple
```

> **Note** `NR==1 {max=$1}` stores the top count; `$1==max` then prints every line with that count, so a tie is never hidden (`head -n 1` would show only one).

DEBUG NOTES · WHAT WENT WRONG FIRST

- `uniq fruits.txt` on its own left duplicates in place: `uniq` only removes *adjacent* repeats, so sort first.
- `grep -ic apple banana grape orange fruits.txt` gave "No such file": grep treats every word after the pattern as a file name. To search several words: `grep -E "apple|banana" fruits.txt`.
- In my first input file apple and banana were tied at 2. `head -n 1` showed only one of them; the awk filter printed both.
- `awk '... 1==$max'` printed nothing: `$max` means "the field numbered max". The variable is plain `max`, and the field is `$1`.

EXERCISE 03

find

Locate files and directories

INPUT FILE

```
mkdir -p testdir/logs
touch testdir/file1.txt
touch testdir/file2.txt
touch testdir/logs/app.log
touch testdir/logs/error.log
```

TASKS

1 Find all .txt files.

```
$ find ./testdir -name "*.txt" | sort
./testdir/file1.txt
./testdir/file2.txt
```

> **Note** `find` lists files in on-disk order, which can differ between machines. `| sort` makes the output repeatable.

2 Find all .log files.

```
$ find ./testdir -name "*.log" | sort
./testdir/logs/app.log
./testdir/logs/error.log
```

3 Count the total number of files.

```
$ find ./testdir -type f | wc -l
4
```

4 Find only directories.

```
$ find ./testdir -type d | sort
./testdir
./testdir/logs
```

DEBUG NOTES · WHAT WENT WRONG FIRST

- `-name "*.logs"` found nothing: the pattern has to match the real extension, `.log`.
- `-name "{.txt, .log}"` found nothing: find does not expand braces. Use `find ./testdir \( -name "*.txt" -o -name "*.log" \)`.
- `find ... | grep -nc ".txt"` counts too, but `| wc -l` is simpler and works for any extension.

EXERCISE 04

cut

Extract columns

INPUT FILE

```
cat > employees.txt << EOF
101:John:Developer
102:Mary:Tester
103:Alex:DevOps
EOF
```

TASKS

1. **Display only IDs.**

```
$ cut -d ':' -f1 employees.txt
```

```
101
102
103
```

2. **Display only names.**

```
$ cut -d ':' -f2 employees.txt
```

```
John
Mary
Alex
```

3. **Display only roles.**

```
$ cut -d ':' -f3 employees.txt
```

```
Developer
Tester
DevOps
```

> **Note** `-d ':'` sets the delimiter; `-f3` selects field 3.

DEBUG NOTES · WHAT WENT WRONG FIRST

- `cut-d ':' -f1` gave “command not found”: a space is needed between the command and its options.

EXERCISE 05

sed

Find-and-replace, print and delete lines

INPUT FILE

```
cat > project.txt << EOF
Linux is good
Linux commands are useful
I like Linux
EOF
```

TASKS

1 Replace Linux with UNIX while displaying output (file unchanged).

```
$ sed 's/Linux/UNIX/g' project.txt
```

```
UNIX is good
UNIX commands are useful
I like UNIX
```

> **Note** `g` replaces every match on a line; without it only the first match per line changes. sed is case-sensitive: `linux` would not match `Linux`.

2 Replace Linux with Ubuntu and save the changes.

```
$ sed -i 's/Linux/Ubuntu/g' project.txt
```

no output

```
$ cat project.txt
```

```
Ubuntu is good
Ubuntu commands are useful
I like Ubuntu
```

> **Note** `-i` edits the file in place, so the next tasks see the Ubuntu version.

3 Print only line 2.

```
$ sed -n '2p' project.txt
```

```
Ubuntu commands are useful
```

> **Note** `-n` turns off automatic printing, so only the line selected by `2p` appears. `awk 'NR==2' project.txt` does the same.

4 Delete line 1 while displaying output.

```
$ sed '1d' project.txt
```

```
Ubuntu commands are useful
I like Ubuntu
```

DEBUG NOTES · WHAT WENT WRONG FIRST

- `sed '2p'` without `-n` printed every line, with line 2 twice.
- `sed -n '1d'` printed nothing: `-n` stops auto-printing and `d` never prints.
- `sed '2'` gave “missing command”: a line address needs an action such as `p`, `d` or `s`.

EXERCISE 06

awk

Fields, conditions and totals

INPUT FILE

```
cat > marks.txt << EOF
John 85
Mary 92
Alex 70
Sarah 95
EOF
```

TASKS

1 Display only names.

```
$ awk '{print $1}' marks.txt
```

```
John
Mary
Alex
Sarah
```

2 Display only marks.

```
$ awk '{print $2}' marks.txt
```

```
85
92
70
95
```

3 Show students scoring above 80.

```
$ awk '$2 > 80 {print $1}' marks.txt
```

```
John
Mary
Sarah
```

4 Calculate total marks.

```
$ awk '{sum += $2} END {print sum}' marks.txt
```

```
342
```

> **Note** The main block runs once per line; `END` runs once after the last line, so the total prints only once.

DEBUG NOTES · WHAT WENT WRONG FIRST

- `awk 'marks=$1+$2+$3+$4 ...'` added fields *across* one row instead of down the marks column.
- `awk '{sum += $2; print sum}'` printed a running total (85, 177, 247, 342). Moving the print into `END` gives just the total.
- A stray `E` in `{sum += $2 E}` happened to work because E is an empty variable; removed.

EXERCISE 07

Process management

Background jobs, finding and killing processes

SETUP (RUN FIRST)

```
sleep 500 &
sleep 600 &
sleep 700 &
```

PIDs are different on every run.

TASKS

1 Find all sleep processes.

```
$ pgrep -a sleep
655 sleep 500
656 sleep 600
657 sleep 700
```

> **Note** `ps -u | grep sleep` also works but lists the grep command itself as a fourth match.

2 Count them.

```
$ pgrep -c sleep
3
```

3 Kill one process.

```
$ kill 655        # PID of 'sleep 500' from task 1
```

no output

```
$ pgrep -c sleep
2
```

4 Kill all sleep processes.

```
$ killall sleep
```

no output

```
$ pgrep -c sleep
0
```

> **Note** `pkill sleep` does the same. `kill $(pgrep sleep)` works too.

DEBUG NOTES · WHAT WENT WRONG FIRST

- `ps -u | grep -ic sleep` returned 4 instead of 3: the `grep sleep` process matched itself. `pgrep` avoids that.
- `ps -u | grep sleep | kill` gave a usage error: kill takes PIDs as arguments, it does not read them from a pipe.

EXERCISE 08

Permissions

chmod in symbolic and octal form

INPUT FILE

```
touch secret.txt
```

TASKS

1 Give the owner full permissions only.

```
$ chmod 700 secret.txt
```

no output

```
$ stat -c '%A %a %n' secret.txt
-rwx------ 700 secret.txt
```

> **Note** Octal digits are owner / group / others, each the sum of r=4, w=2, x=1. So 7 = rwx and 0 = nothing.

2 Give everyone read permission.

```
$ chmod a+r secret.txt
```

no output

```
$ stat -c '%A %a %n' secret.txt
-rwxr--r-- 744 secret.txt
```

3 Remove write permission for group and others.

```
$ chmod go-w secret.txt
```

no output

```
$ stat -c '%A %a %n' secret.txt
-rwxr--r-- 744 secret.txt
```

> **Note** No visible change here because group and others had no write bit after tasks 1–2. `-w` only removes; it never adds.

4 Make the file executable only for the owner.

```
$ chmod u+x,go-x secret.txt
```

no output

```
$ stat -c '%A %a %n' secret.txt
-rwxr--r-- 744 secret.txt
```

> **Note** `u+x` alone adds execute for the owner but leaves others untouched; `go-x` guarantees “only”.

DEBUG NOTES · WHAT WENT WRONG FIRST

- `stat -c '%A %a %n'` prints the permission string, the octal value and the name. It is used here instead of `ls -l` so the output does not depend on user name or date.

EXERCISE 09

Pipelines

Chaining commands

INPUT FILE

```
cat > names.txt << EOF
John
Alex
John
Mary
Alex
Alex
David
EOF
```

TASKS

1 Sort the names.

```
$ sort names.txt
```

```
Alex
Alex
Alex
David
John
John
Mary
```

2 Remove duplicates.

```
$ sort -u names.txt
```

```
Alex
David
John
Mary
```

3 Count each name's occurrences.

```
$ sort names.txt | uniq -c
```

```
      3 Alex
      1 David
      2 John
      1 Mary
```

4 Display the most frequent name using a pipeline.

```
$ sort names.txt | uniq -c | sort -nr | awk 'NR==1 {max=$1} $1==max {print $2}'
```

```
Alex
```

> **Note** `sort -nr` puts the highest count first; the awk filter prints only the name(s) with that count.

DEBUG NOTES · WHAT WENT WRONG FIRST

- The first answer, `sort | uniq -c | head -n 1`, printed `alex` only because “alex” sorts first alphabetically. It looked right for the wrong reason; `sort -nr` fixes the logic.
- A trailing space (`john␣` vs `john`) made uniq count the same name twice. Invisible whitespace in input data is a classic source of mismatches.

EXERCISE 10

Log analysis

The most realistic one

INPUT FILE

```
cat > app.log << EOF
INFO Application Started
ERROR Database Failed
INFO User Login
ERROR Connection Lost
WARNING Low Memory
ERROR Timeout
INFO Logout
EOF
```

TASKS

1 Count total ERROR entries.

```
$ grep -c "ERROR" app.log
3
```

> **Note** Matched case-sensitively on purpose: log severities are uppercase, and `-i` could also hit words like “error” inside a message.

2 Show only WARNING lines.

```
$ grep "WARNING" app.log
WARNING Low Memory
```

3 Display the last 3 lines.

```
$ tail -n 3 app.log
WARNING Low Memory
ERROR Timeout
INFO Logout
```

4 Display line numbers with ERROR messages.

```
$ grep -n "ERROR" app.log
2:ERROR Database Failed
4:ERROR Connection Lost
6:ERROR Timeout
```

5 Count total INFO entries.

```
$ grep -c "INFO" app.log
3
```

DEBUG NOTES · WHAT WENT WRONG FIRST

- The first input file was missing the `ERROR Timeout` line, so the count came out as 2. Check the stimulus before debugging the checker.

EXERCISE 11

Mini challenge: sales report

Using only awk, sort, head and pipes

INPUT FILE

```
cat > sales.txt << EOF
John 5000
Mary 7000
Alex 3000
Sarah 9000
David 6000
EOF
```

TASKS

1 Find the highest sale.

```
$ sort -k2,2nr sales.txt | awk 'NR==1 {max=$2} $2==max'
Sarah 9000
```

> **Note** `-k2,2nr` = sort on column 2 only, numerically, in reverse. The awk filter keeps all ties.

2 Find the lowest sale.

```
$ sort -k2,2n sales.txt | awk 'NR==1 {min=$2} $2==min'
Alex 3000
```

3 Calculate total sales.

```
$ awk '{sum += $2} END {print sum}' sales.txt
30000
```

4 Calculate average sales.

```
$ awk '{sum += $2} END {print sum/NR}' sales.txt
6000
```

> **Note** Inside `END`, `NR` holds the total number of lines, so the average adapts to any file size.

5 Display the top 3 performers.

```
$ sort -k2,2nr sales.txt | head -n 3
Sarah 9000
Mary 7000
David 6000
```

DEBUG NOTES · WHAT WENT WRONG FIRST

- `sort -n sales.txt` sorted by name: without `-k`, sort compares the whole line from column 1.
- `sort -k2, 2nr` (with a space) gave "invalid number": the key spec must be one word, `-k2,2nr`.
- `awk '{sum+=%2}'` gave a syntax error: fields are `$2`, not `%2`.
- The first average hard-coded the count (`sum/5`); `sum/NR` works for any number of lines.